# 12. Bölüm

# Tip Dönüşümleri ve Lamda İfadeleri

## 12.1 Tip Dönüşümü Nedir?

Tip dönüşümü, bir veri tipini anlamını kaybetmeyecek şekilde başka uyumlu tipe dönüştürmek anlamına gelir.

```
1 #include <iostream>
2 int main() {
3     int i = 10;
4     char c = 'A';
5     std::cout << c << " karakterinin ASCII kodu:" << (int)c << std::endl; // A karakterinin ASCII
      ↪ kodu:65
6     int toplam = i + c; /* c karakteri "char" "veri tipinden "int" veritipine dönüştürüldü ve i
      ↪ değişkeni ile toplandı ardından atama işleci ile int veri tipindeki toplam değişkenine
      ↪ atandı */
7     std::cout << "Toplam:" << toplam; //75
8 }
```

Yukarıda verilen kodda iki tane tip dönüşümü yapılmıştır;

1. 5. Satırda konsola karakter kodunu yazdırmak için (`(int)c`) ifadesi ile **kasıtlı tip dönüşümü** (explicit type casting) yapılmıştır.
2. 7. Satırda ise `i + c` ifadesinde `int` veri tipinde `i` değişkenini `char` veri tipinde `c` değişkeniyle toplamak için `c` değişkeni bellekte daha fazla yer kaplayan `int` veri tipine **üstü kapalı tip dönüşümü** (imlpicit type casting) yapılır. Daha sonra toplama işlemi yapılır.

## 12.2 Üstü Kapalı Tip Dönüşümleri

C++ dilinde **üstü kapalı veri tipi dönüşümü** (implicit type casting), **zorlama** (coercioncoercion), **standart tip dönüşümü** (standart type casting) veya **otomatik tip dönüşümü** (automatic type conversion) olarak da bilinir. Bu tip dönüşümünde programcının açık talimatlar vermesine gerek kalmadan derleyici bir veri tipini otomatik olarak başka bir uyumlu veri tipine dönüştürür. Şu durumlarda gerçekleşir;

- ◇ Dönüşüm farklı veri tiplerinin değerleri üzerinde gerçekleştirilir.
- ◇ Farklı bir veri tipi bekleyen bir fonksiyona bir argüman geçirmeniz halinde gerçekleşir.
- ◇ Bir veri tipinin değerini başka bir veri tipinin değişkenine atama durumunda gerçekleşir.

> Üstü kapalı tip dönüşümde bellekte az yer kaplayan veri tipinden tanımlanmış değişkenler, bellekte çok yer kaplayan değişkenlere atanmaya çalışıldığında otomatik olarak çok yer kaplayanın veri tipine dönüştürülür (widening casting).
>
> `bool`→`char`→`short int`→`int`→`unsigned int`→`long`→`unsigned`→`long long`→`float`→`double`→`long double`

```
int main() {
    char c;   //1 bayt
    int i;    //4 bayt
    long l;   //8 bayt
    float f;  //4 bayt
    double d; //8 bayt
    //...
    c=65; // 65 int değişmezi char veritipine dönüştürürlür.
    i=c;  //int veri tipine atanmadan önce; char, int veritipine dönüştürülür.
    l=i;  //long veri tipine atanmadan önce; int, long veritipine dönüştürülür.
    f=12.50f; // 12.50 float değişmez olduğundan tip dönüşümü yapılmaz.
    d=f; //double veri tipine atanmadan önce; float, double veritipine dönüştürülür.
    //...
}
```

Ayrıca Üstü kapalı tip dönüşümde bellekte çok yer kaplayan veri tipinden tanımlanmış değişkenler, bellekte az yer kaplayan değişkenlere atanmaya çalışıldığında, tip dönüşümü otomatik olarak yapılır ancak **veri kaybı söz konusu olur** (narrowing casting).

```
int main() {
    char c;
    int i;
    float f;
    i=4095; //i=0x00000FFF -> 0x00,0x00,0x0F,0xFF olarak 4 bayt
    c=i; /* c=0xFF -> c değişkenine int veritipinden char veritipine en anlamsız baytı atanır.*/
    f=12.50f;
    /* Kayan noktalı bir sayıdan tam sayıya atama yapılıyor ise tam kısmı tam sayıya çevrilir ve
     ↪ sonrasında atama yapılır. */
    i=f; //i=12
    //...
}
```

## 12.3 Bilinçli Tip Dönüşümü

Bazen belli işlemleri daha kontrollü yapmak amacıyla programcı yapılan işlemde işlenen veri tipini bir başka ver tipine kasıtlı olarak değiştirir. Bu tür dönüşümüne **bilinçli tür dönüşümü** (explicit type casting) denir.

C dilinde **tekli tip dönüştürme işleci** (unary cast operator) ile yapılır.Diğer tekli işleçler gibi ifadenin (expression) önünde kullanılır ve parantez içerisinde dönüştürülmek istenen veri tipi yazılarak `int j=(int) 12.8/4;` şeklinde kodlanır. Bu dönüşümler C dilinden gelen bilinçli dönüştürmedir.

### §12.3.1 Derleme Zamanı Kasıtlı Tip Dönüşümü

Modern C++'ta daha güvenli ve ne yapıldığı daha net gösteren `static_cast` işleci (static_cast operator) tercih edilir. Bu da derleme zamanında (compile-time) hataların yakalanmasına izin verir. Yani derleme zamanı tip dönüşümleri için kullanılır. Aşağıda buna ilişkin bir örnek program verilmiştir;

```
#include <iostream>
int main() {
    float x = 9.9f; // 'f' soneki değişmezin bir float olduğunu netleştirir
    int y = x + 3.3; /* Otomatik (Implicit) dönüşüm: 9.9 + 3.3 = 13.2, int'e atanırken 13 olur. */

    int z = static_cast<int>(x) + 3.3; /* C++ Tarzı Kasıtlı (Explicit) dönüşüm: static_cast
     ↪ kullanımı: Önce x tam sayıya (9) çekilir, sonra 3.3 eklenir (12.3), int'e atanırken 12
     ↪ olur. */

    double d = 1.2;
    int s = static_cast<int>(d + 1); /* C++ Tarzı Kasıtlı (Explicit) dönüşüm: static_cast
     ↪ kullanımı: Önce d tam sayıya (1) çekilir, sonra 1 tamsayısı eklenir (2), int'e atanırken
     ↪ 2 olur. */

    std::cout << "x: " << x << std::endl;  // Çıktı: 9.9
    std::cout << "y: " << y << std::endl;  // Çıktı: 13
    std::cout << "z: " << z << std::endl;  // Çıktı: 12
    std::cout << "s: " << s << std::endl;  // Çıktı: 2
}
```

### §12.3.2 Dinamik Dönüşüm

Kasıtlı tip dönüşümünde **dinamik dönüşüm** (dynamic cast), çok biçimlilik (polymorphism) ve kalıtımda (inheritance) çalışma zamanı (run-time) tip dönüşümü için kullanılır. Alt sınıfı üst sınıf veri tipine dönüştürmek için `dynamic_cast` işleci (dynamic_cast operator) kullanılır.

```
#include <iostream>
class Taban {
    public:
    virtual ~Taban() = default;
    virtual void fonksiyon() {
```

```
        std::cout << "Taban sinif fonksiyonu..." << std::endl;
    }
};
class Turemis : public Taban {
    public:
    // 'override' anahtar kelimesi hatayı önler ve okunabilirliği artırır
    void fonksiyon() override {
        std::cout << "Turemis sinif fonksiyonu geçersiz kılınmış!" << std::endl;
    }
};
int main() {
    // 1. Nesne oluşturma
    Taban* b = new Turemis();
    // 2. dynamic_cast: Taban işaretçisini Türemiş işaretçisine güvenli şekilde çevirir
    Turemis* d = dynamic_cast<Turemis*>(b);
    if (d != nullptr) { // C++11 sonrası nullptr kullanımı standarttır
        std::cout << "Donusturme basarili! d nesnesi uzerinden cagri yapiliyor:" << std::endl;
        d->fonksiyon(); // Artık Türemiş sınıfın fonksiyonu çalışır
    } else {
        std::cout << "Donusturme basarisiz!" << std::endl;
    }
    // 3. Bellek temizliği (Modern C++'ta bunu delete ile yapmak zorundayız)
    delete b;
}
```

### §12.3.3 Sabit Dönüşüm

Kasıtlı tip dönüşümünde **sabit dönüşüm** (const cast), `const` veya `volatile` niteleyicilerini kaldırır veya ekler. Bunun için `const_cast` işleci (const_cast operator) kullanılır.

```
#include <iostream>
class Kisi {
    private:
    int yas;
    public:
    Kisi(int pYas) : yas(pYas) {}
    // ÖNEMLİ: Bir fonksiyonun amacı veriyi değiştirmekse, o fonksiyon 'const' olmamalıdır.
    // Ancak eğitim amaçlı const_cast kullanacaksanız:
    void yasDegistir() const {
        // 'this' üzerindeki const özelliğini kaldırarak yas alanına erişim sağlar
        const_cast<Kisi*>(this)->yas = 30;
    }
    // Getter fonksiyonları veriyi değiştirmediği için 'const' olmalıdır
    int getYas() const {
        return yas;
    }
};
// Fonksiyon parametresi değiştirilmeyecekse const int* tercih edilir, ancak burada toplama yapıp
↪ döndüğü için int* kalabilir.
int fonksiyon(int* ptr) {
    return (*ptr + 100);
}
int main() {
    // 1. Nesne işlemleri
    Kisi ilhan(50);
    std::cout << "Eski Yas: " << ilhan.getYas() << std::endl;
    ilhan.yasDegistir();
    std::cout << "Yeni Yas: " << ilhan.getYas() << std::endl;
    // 2. Sabit (const) bir değişken üzerinde const_cast kullanımı
    const int val = 500;
    const int* ptr = &val;
    // Bir const int* değişkenini, int* bekleyen bir fonksiyona göndermek için:
    int* ptr1 = const_cast<int*>(ptr);
```

```
    std::cout << "Fonksiyon Sonucu: " << fonksiyon(ptr1) << std::endl;
}
/*Program Çıktısı:
Eski Yas: 50
Yeni Yas: 30
Fonksiyon Sonucu: 600
*/
```

### §12.3.4 Yeniden Yorumlama Dönüşümü

Kasıtlı tip dönüşümünde **yeniden yorumlama dönüşümü** (reinterpret cast), dönüştürme öncesi ve sonrası veri tipleri farklı olsa bile, bir veri tipindeki göstericiyi başka bir veri tipindeki göstericiye dönüştürmek için kullanılır. Gösterici tipinin ve göstericinin işaret ettiği verinin aynı olup olmadığı kontrol edilmez. Kısaca bu operatör, bellekteki ham bitleri (raw bits) bir tipten tamamen farklı bir tipe zorla yorumlamak için kullanılır. Bunun için `reinterpret_cast` işleci (reinterpret_cast operator) kullanılır.

```
#include <iostream>
int main() {
    // 1. Bellekte bir tam sayı (int) için yer aç ve 65 değerini ata.
    // 65 sayısı ASCII tablosunda 'A' harfine karşılık gelir.
    int* p = new int(65);
    // 2. reinterpret_cast: int* adresini char* olarak yorumla.
    // Bellekteki verinin bitlerine dokunmaz, sadece bakış açısını değiştirir.
    char* ch = reinterpret_cast<char*>(p);
    std::cout << "Integer Degeri (*p): " << *p << std::endl; // Integer Degeri (*p): 65
    std::cout << "Karakter Degeri (*ch): " << *ch << std::endl; // Karakter Degeri (*ch): A
    std::cout << "Integer Adresi (p): " << p << std::endl; // Integer Adresi (p): 0x2166a2b0
    // 3. char* yazdırırken adres görmek istiyorsak void*'a çevirmeliyiz.
    // Aksi halde cout bunu bir metin (string) sanıp yanındaki bellekleri de okur.
    std::cout << "Karakter Adresi (ch): " << static_cast<void*>(ch) << std::endl; //Karakter
     ↪  Adresi (ch): 0x2166a2b0
    // Bellek temizliği (new kullanıldığı için şart)
    delete p;
}
```

### §12.3.5 Kasıtlı Tip Dönüşümü Örneği

Bir tam sayının hangi 4 bayt bellek alanından oluştuğu 4.3 başlığında anlatılmıştı. Aşağıda, tanımlanan bir tam sayının hangi baytlardan oluştuğunu gösteren bilinçli tip dönüşümlerini içeren program örneği verilmiştir;

```
#include <iostream>
#include <iomanip> // hex, dec ve setfill için
int main() {
    // 0x10203040 sayısı belleğe bayt bayt yerleşir.
    int i = 0x10203040;
    // int* adresini char* olarak yorumluyoruz (Explicit Cast)
    char* p = reinterpret_cast<char*>(&i);
    // Genel bilgi
    std::cout << "Sayi (Onluk): " << std::dec << i << std::endl;
    std::cout << "Sayi (Onaltilik): 0x" << std::hex << i << std::endl;
    std::cout << "--------------------------" << std::endl;
    // Baytları tek tek inceleyelim
    // static_cast<int> kullanıyoruz çünkü char doğrudan cout'a giderse
    // ASCII karşılığını basmaya çalışır, biz sayısal değerini istiyoruz.
    for (int n = 0; n < 4; ++n) {
        std::cout << n + 1 << ". bayt (Adres: " << static_cast<void*>(p + n) << "): "
        << "Onluk: " << std::dec << static_cast<int>(*(p + n)) << " \t"
        << "Onaltilik: 0x" << std::hex << static_cast<int>(*(p + n)) << std::endl;
    }
}
/*Program Çıktısı:
Sayi (Onluk): 270544960
Sayi (Onaltilik): 0x10203040
--------------------------
```

```
1. bayt (Adres: 0x7ffc89aaf71c): Onluk: 64     Onaltilik: 0x40
2. bayt (Adres: 0x7ffc89aaf71d): Onluk: 48     Onaltilik: 0x30
3. bayt (Adres: 0x7ffc89aaf71e): Onluk: 32     Onaltilik: 0x20
4. bayt (Adres: 0x7ffc89aaf71f): Onluk: 16     Onaltilik: 0x10
*/
```

## 12.4 Tip Dönüşümünün Üstünlük ve Zayıflıkları

Tip dönüşümünün üstün yönleri;

- **İşlemlerde Esneklik**: Farklı veri tiplerini içeren işlemleri gerçekleştirmede esneklik sağlar. Programcının verileri bir tipten diğerine açıkça dönüştürmesini sağlayarak karışık tip aritmetiğini ve diğer işlemleri kolaylaştırır.
- **Uyumluluk**: Farklı veri tipleri arasında uyumluluğun sağlanmasına yardımcı olur. Programcının verileri belirli bir bağlamda kullanmadan önce uyumlu bir tipe dönüştürmesini sağlayarak veri uyumsuzluğu hatalarını önler.
- **Hassasiyet Kontrolü**: Hassasiyet kontrolünün kritik olduğu durumlarda, programcının özellikle sayısal işlemlerde veri tipleri arasında dönüşüm yaparak istenen hassasiyeti açıkça belirtmesini sağlar.
- **Açıklık**: Tip dönüşümü, programcının veri tipini değiştirme niyetini belirterek kodu daha açık hale getirir. Bu, kod okunabilirliğini artırabilir ve işlenen veri tipiyle ilgili kafa karışıklığını azaltabilir.

Tip dönüşümünün zayıf yönleri;

- **Hassasiyet Kaybı**: Tip dönüştürmenin en büyük dezavantajlarından biri hassasiyet kaybı potansiyelidir. Örneğin, kayan noktalı bir sayıyı tam sayıya dönüştürürken kesirli kısım kesilir ve bu da bilgi kaybına yol açar.
- **Çalışma Zamanı Yükü**: Bilinçli tip dönüştürme genellikle program yürütme sırasında dönüştürmenin gerçekleştirilmesi gerektiğinden çalışma zamanı yüküne neden olur. Bu ek işlem, özellikle tip dönüştürmenin sık olduğu durumlarda performansı etkileyebilir.
- **Derleyici Uyarıları ve Hataları**: Yanlış veya güvenli olmayan tip dönüştürme, derleyici uyarılarına veya hatalarına yol açabilir. Örneğin, uyumsuz tipler arasında dönüştürme yapmaya çalışmak veya geçersiz dönüştürme sözdizimi kullanmak, hata ayıklaması zor olabilecek sorunlara yol açabilir.
- **Tanımsız Davranış Potansiyeli**: Bazı durumlarda, tip dönüştürme, özellikle uyumsuz tipler arasında dönüştürme yaparken veya dönüştürülen değer hedef tipin aralığının dışında olduğunda tanımsız davranışa yol açabilir. Bu, programda öngörülemez durumlara neden olabilir.
- **Kod Bakım Zorlukları**: Bilinçli tip dönüşümüne kodun bakımı ve anlaşılması, özellikle karmaşık veya iç içe ifadelerde gerçekleştirildiğinde, daha zor hale gelebilir. Bu, artan kod karmaşıklığına ve azalan okunabilirliğe yol açabilir.
- **Taşınabilirlik Endişeleri**: Bilinçli tip dönüşümüne farklı platformlar veya derleyiciler arasında daha az taşınabilir olabilir. C'de tip dönüşümünün davranışı değişebilir ve belirli veri tipi boyutları hakkındaki varsayımlar tüm ortamlarda geçerli olmayabilir.

## 12.5 Tip Çıkarımı

**Tip çıkarımı** (type inference), bir ifadenin veri tipinin otomatik olarak belirlenmesi anlamına gelir. C++ dilinin 2011 sürümünden sonra, `auto` ve `decltype` gibi anahtar sözcükler bu amaçla dile dahil edilmiştir. Böylece bir programcının tür çıkarımını derleyicinin kendisine bırakmasına olanak sağlıyor. Tür çıkarımı yetenekleriyle, derleyicinin zaten bildiği şeyleri yazmak gerekmez. Tüm tipler yalnızca derleyici aşamasında belirlendiği için, derleme için gereken süre biraz artar ancak programın çalışma süresini etkilemez.

C++ dilinde `auto` anahtar sözcüğü (auto keyword), bildirilen değişkenin tipinin ilk değer verme işleminden otomatik olarak çıkarır. Fonksiyonlar söz konusu olduğunda, dönüş tipleri `auto` ise çalışma zamanında (run-time) bu dönüş tipi belirlenir.

```
#include <iostream>
#include <typeinfo> // typeid kullanımı için gerekli
int main() {
    // auto kullanımı: Derleyici sağdaki değere bakarak tipi belirler.
    // auto a; // HATA! İlk değer atanmadığı için tip belirlenemez.
    auto x = 4;           // int
    auto y = 3.37;        // double (varsayılan)
    auto z = 3.37f;       // float ('f' soneki nedeniyle)
    auto c = 'a';         // char
    auto ptr = &x;        // int* (pointer to int)
    auto pptr = &ptr;     // int** (pointer to a pointer)
```

```
    // typeid().name() çıktıları derleyiciye (GCC, Clang, MSVC) göre değişir.
    // i = int, d = double, f = float, c = char, Pi = pointer to int vb.
    std::cout << "Degisken Tipleri:" << std::endl;
    std::cout << "x    : " << typeid(x).name()    << " (Değer: " << x << ")" << std::endl;
    std::cout << "y    : " << typeid(y).name()    << " (Değer: " << y << ")" << std::endl;
    std::cout << "z    : " << typeid(z).name()    << " (Değer: " << z << ")" << std::endl;
    std::cout << "c    : " << typeid(c).name()    << " (Değer: " << c << ")" << std::endl;
    std::cout << "ptr  : " << typeid(ptr).name()  << " (Adres: " << ptr << ")" << std::endl;
    std::cout << "pptr : " << typeid(pptr).name() << " (Pointer Adresi: " << pptr << ")" <<
     ↪  std::endl;
}
/*Program Çıktısı:
Degisken Tipleri:
x    : i (Değer: 4)
y    : d (Değer: 3.37)
z    : f (Değer: 3.37)
c    : c (Değer: a)
ptr  : Pi (Adres: 0x7ffe2d5453f4)
pptr : PPi (Pointer Adresi: 0x7ffe2d5453e8)
*/
```

C++ dilinde `auto` anahtar sözcüğü belirli bir tipe sahip bir değişken bildirimine olanak tanırken, `decltype` ifadeden (expression) tip çıkarmanıza olanak tanır; dolayısıyla `decltype`, geçirilen ifadenin tipini değerlendiren bir **tip işlecidir** (decltype operator). Kısaca `auto` bir değerden tip çıkarırken, `decltype` **bir ifadeden veya fonksiyondan tip çıkarır**.

```
#include <iostream>
#include <typeinfo> // typeid ve name() metodu için gerekli
// Örnek fonksiyonlar
int fun1() { return 10; }
char fun2() { return 'g'; }
int main() {
    // 1. Fonksiyon dönüş tiplerinden değişken tanımlama
    // fun1() çağrılmaz, sadece dönüş tipine (int) bakılır.
    decltype(fun1()) x = 0;
    // fun2() dönüş tipine (char) bakılır.
    decltype(fun2()) y = 'a';
    std::cout << "x degiskeninin tipi: " << typeid(x).name() << " (int)" << std::endl;
    std::cout << "y degiskeninin tipi: " << typeid(y).name() << " (char)" << std::endl;
    // 2. Mevcut bir değişkenden tip kopyalama
    char a = 65; // 'A'
    // b'nin tipi a ile aynı (char) olur.
    // Dikkat: a + 5 işleminin sonucu int olsa bile decltype(a) zorlayıcıdır.
    decltype(a) b = a + 5;
    std::cout << "b degiskeninin tipi: " << typeid(b).name() << " (char)" << std::endl;
    std::cout << "b degiskeninin degeri: " << b << " (ASCII 70)" << std::endl;
}
/*Program Çıktısı:
x degiskeninin tipi: i (int)
y degiskeninin tipi: c (char)
b degiskeninin tipi: c (char)
b degiskeninin degeri: F (ASCII 70)
*/
```

## 12.6 Lamda İfadeleri

Bir **lamda ifadesi** (lambda expression), basit fonksiyon nesneleri oluşturmak için özlü bir yol sağlar. **Lamda** adı, *Alonzo Church* tarafından 1930'larda mantık ve hesaplanabilme hakkındaki soruları araştırmak için icat edilen matematiksel bir form olan lamda hesabından gelir. Lamda hesabı, fonksiyonel bir programlama dili olan LISP'in temelini oluşturmuştur.

C++14 ile hayatımıza giren lamda ifadesi genellikle çağrılabilir bir nesne alan fonksiyonlara argüman olarak kullanılır. Lamda ifadesi, gövdesi tanımlı bir fonksiyon oluşturmaktan daha basit olabilir. Lamda ifadeleri fonksiyon nesnelerini **satır içi** (inline) tanımlamaya izin verirler.

Söz dizimi aşağıda verilen lamda ifadesi tipik olarak dört bölümden oluşur;

```
[yakalama-listesi] (parametreler) -> donus_tipi { fonksiyon-govdesi }
```

1. **[] Yakalama Listesi** (capture clause): Lambdanın içinde olduğu bloktaki yerel değişkenlere nasıl erişeceğini belirler. Hiçbir yerel değişkene erişilmeyecek ise boş bırakılabilir.
2. **() Parametre Listesi** (parameters): Normal fonksiyonlardaki gibi argüman listesidir. Parametre yoksa boş bırakılabilir.
3. **-> Dönüş tipi** (return type): Dönüş tipidir. Çoğu zaman derleyici bunu tip çıkarımı (type inference) yaparak otomatik olarak belirler, bu yüzden yazılması zorunlu değildir.
4. **{} Gövde** (body): Fonksiyonun yapacağı işlerin yazıldığı bloktur. Yapılacak bir şey yoksa boş bırakılabilir.

### §12.6.1 Yakalama Listesi

Varsayılan olarak köşeli parantez içindeki `[]` yakalama listesi, içinde olduğu kapsamın değişkenlerine bir lamda tarafından erişilemez. Bir değişkeni yakalamak, onu lamda içinde, bir kopya veya bir referans olarak erişilebilir hale getirir. Yakalanan değişkenler lamdanın bir parçası haline gelir; fonksiyon argümanlarının aksine, lamda çağrılırken parametre gibi geçirilmeleri gerekmez. Yakalama listesi, lambdanın çevresindeki değişkenleri "içeriye" nasıl alacağını belirler.

1. `[x]` (**Değere Göre Belirli Değişkeni Yakalama**): Sadece `x` değişkenini kopyalayarak içeri alır. Lamda içinde `x` değişkeninin kopyasıyla çalışılır, dışarıdaki orijinal `x` değişmez.
2. `[=]` (**Değere Göre Her Şeyi Yakalama**): Kapsamdaki (scope) tüm yerel değişkenleri kopyalayarak içeri alır. Güvenlidir çünkü orijinal veriler korunur, ancak çok sayıda büyük değişkenlerin kopyalama maliyeti oluşabilir.
3. `[&]` (**Referans ile Her Şeyi Yakalama**): Kapsamdaki tüm yerel değişkenlere referans (adres) yoluyla erişir. Lamda içinde yapılan değişiklikler dışarıdaki orijinal değişkeni de değiştirir. Çok hızlıdır (kopyalama yapmaz) ama değişkenin ömrü biterse hata riski taşır.

```
#include <iostream>
int main() {
    int i = 10;
    //auto lamda1 = []() { return a*9; }; // Hata: 'a' yakalanmadığı için erişilemez
    auto lamda2 = [i]() { return i*9; }; // 'a' değerinden yakalanır
    std::cout << "lamda2:" << lamda2() << std::endl; //lamda2:90
    auto lamda3 = [&i]() { return ++i; }; // 'a' referansından yakalanır
    std::cout << "lamda3:" << lamda3() << std::endl; //lamda3:11
    int a=10,b=20;
    // [a] -> Sadece a'yı kopyalar
    auto lamdaA = [a]() {
        // a = 30; // HATA! Kopyalanan değer varsayılan olarak değiştirilemez (read-only).
        std::cout << "Sadece a: " << a << std::endl;
    };
    // [=] -> a ve b'yi kopyalar
    auto lamdaHepsi = [=]() {
        // a = 30; // HATA! Kopyalanan değer varsayılan olarak değiştirilemez (read-only).
        std::cout << "Kopyalanan a+b: " << a + b << std::endl;
    };
    // [&] -> a ve b'ye referansla erişir
    auto lamdaReferansHepsi = [&]() {
        a = 100; // Dışarıdaki 'a' değişkenini doğrudan değiştirir!
    };
    lamdaReferansHepsi();
    std::cout << "Orijinal a artik: " << a << std::endl; // Çıktı: 100
    lamdaHepsi(); // Çıktı: 30 (ReferansHepsi'den önce a=10, b=20 idi.)
}
```

### §12.6.2 Parametre Listesi

Lamda fonksiyonlarında parantezler `()` parametre listesidir ve normal fonksiyonlardaki ile hemen hemen aynıdır. Lamda hiçbir argüman almazsa, bu parantezler atlanabilir (tabiki lamdayı değiştirilebilir olarak bildirmeniz gerekmediği sürece).

```
#include <iostream>
struct Sinif {
    void yontem() const {
```

```
        std::cout << "Nesne::yontem() fonksiyonu calisti!" << std::endl;
    }
};
int main() {
    int a = 100;
    Sinif nesne; // Lambdalar içinde kullanılacak nesne
    // 1. Lambda Tanımlama: [capture](parameters){body}
    // nesne nesnesini içeriye 'değer' olarak (kopyalayarak) alır.
    auto call_func1 = [nesne]() {
        nesne.yontem();
    };
    // 2. Parametre parantezleri boşsa opsiyoneldir (C++11 ve sonrası)
    auto call_func2 = [nesne] {
        nesne.yontem();
    };
    // 3. Değişkeni yakalayan bir lambda örneği (call_b için)
    auto lambda_b = [a]() {
        return a + 50;
    };

    int call_b = lambda_b(); // Lambda fonksiyonunu çağırıyoruz
    // Fonksiyonları çağıralım
    call_func1();
    call_func2();

    std::cout << "call_b degiskeninin degeri (a + 50): " << call_b << std::endl;
}
/*Program Çıktısı:
Nesne::yontem() fonksiyonu calisti!
Nesne::yontem() fonksiyonu calisti!
call_b degiskeninin degeri (a + 50): 150
*/
```

Parametre listesi gerçek tipler yerine yer tutucu tipi olan `auto` tipini kullanabilir. Bunu yaparak, bu argüman bir fonksiyon şablonunun şablon (template) parametresi gibi davranır. Aşağıdaki lamdalar, genel kodda bir vektörü (vector) sıralamak istediğinizde eşdeğerdir. Şablon ve dinamik dizi olan vektörü kavramını iletide anlatacağız.

```
#include <iostream>
#include <vector>
#include <algorithm> // std::sort için gerekli
int main() {

    std::vector<int> sayilar = {45, 12, 89, 33, 7};
    std::vector<std::string> kelimeler = {"C++", "Python", "Java", "Assembly"};
    // 1. C++11 Tarzı Lambda: Tip açıkça belirtilmelidir (Örn: int)
    auto sort_cpp11 = [](const int& lhs, const int& rhs) {
        return lhs < rhs;
    };
    // 2. C++14 Tarzı (Generic) Lambda: 'auto' sayesinde her tipi kabul eder
    auto sort_cpp14 = [](const auto& lhs, const auto& rhs) {
        return lhs < rhs;
    };
    // --- Uygulama ---
    // Sayıları C++11 lambdası ile sıralayalım
    std::sort(sayilar.begin(), sayilar.end(), sort_cpp11);
    // Kelimeleri C++14 generic lambdası ile sıralayalım (Aynı lambda int için de çalışırdı!)
    std::sort(kelimeler.begin(), kelimeler.end(), sort_cpp14);
    // Sonuçları Yazdıralım
    std::cout << "Siralanmis Sayilar: ";
    for(int n : sayilar)
        std::cout << n << " ";
    std::cout << std::endl <<"Siralanmis Kelimeler: ";
```

```
    for(const auto& s : kelimeler)
        std::cout << s << " ";
}
/*Program Çıktısı:
Siralanmis Sayilar: 7 12 33 45 89
Siralanmis Kelimeler: Assembly C++ Java Python
*/
```

### §12.6.3 Gövde

Lamda ifadelerinde süslü parantez içinde yer alan gövde, normal fonksiyonlardaki ile aynı olan gövdedir. Bir lamda ifadesinin sonuç nesnesi için, diğer fonksiyon nesnelerinde olduğu gibi `return` talimatı kullanılır.

```
#include <iostream>
int main() {
    int multiplier = 5;
    // [&multiplier] -> multiplier değişkenine bir referans (adres) tutar.
    auto timesDynamic = [&multiplier](int a) {
        return a * multiplier;
    };
    std::cout << "Ilk sonuc: " << timesDynamic(2) << std::endl; // 2 * 5 = 10
    multiplier = 15; // Orijinal değişkeni güncelliyoruz
    // Lambda referans tuttuğu için güncel değeri görür.
    std::cout << "Guncel sonuc: " << timesDynamic(2) << std::endl; // 2 * 15 = 30
}
```

Bazı durumlarda varsayılan olarak, bir lamda ifadesinin dönüş tipi türetilir. Aşağıdaki durumda dönüş tipi `bool` olur.

```
auto f=[](){ return true; };
```

Aşağıdaki söz dizimini kullanarak dönüş tipi belirtebilirsiniz:

```
auto f=[]() -> bool { return true; };
```

Dönüş tipi belirlenirken şu durumlar göz önüne alınmalıdır;

```
#include <iostream>
int main() {
    auto d1 = [](int value) {
        return value > 10; // Return bool
    };
    auto d2 = [](int value) {
        if (value < 10) {
            return 1; // Hata: lamda dönüş tipini belirleyemedi int?
        } else {
            //return 1.5; // Hata: lamda dönüş tipini belirleyemedi double?
        }
    };
    auto d3 = [](int value) -> double {// Dönüş tipi 'double'
        if (value < 10) {
            return 1;
        } else {
            return 1.5;
        }
    };
}
```

### §12.6.4 Lamda Örnekleri

Lamdadaki değerle yakalanan nesneler varsayılan olarak değişmez nesnelerdir (immutable object). Bunun nedeni, oluşturulan kapatma nesnesinin varsayılan olarak const olmasıdır.

```
auto func = [c = 0](){++c; std::cout << c;}; /* ++c lamda'nın durumunu değiştirmeye çalıştığı
↪ için derlenemez. */
```

Değiştirilmeye, `mutable` anahtar sözcüğü kullanılarak izin verilebilir;

```
auto func1 = [c = 0]() mutable {++c; std::cout << c;};
auto func2 = [c = 0]() mutable -> int {++c; std::cout << c; return c;};
```

Aşağıda ana fonksiyonun lamda olarak düzenlenmiş haline bir örnek verilmiştir;

```
#include <iostream>
#include <iomanip> // setw() için gerekli
// C++11 Trailing Return Type kullanımı (auto ... -> int)
auto main() -> int
{
    // Sabitleri 'const' veya 'constexpr' ile tanımlamak iyi bir pratiktir
    int constexpr satir = 3;
    int constexpr sutun = 4;

    // Matris tanımı
    int const matris[satir][sutun] =
    {
        { 1,  2,  3,  4},
        { 5,  6,  7,  8},
        { 9, 10, 11, 12}
    };

    // Matrisi ekrana yazdıran döngüler
    for (int y = 0; y < satir; ++y) {
        for (int x = 0; x < sutun; ++x) {
            std::cout << std::setw(4) << matris[y][x];
        }
        std::cout << std::endl;
    }
}
```

Bir başka örnek olarak klavyeden eleman sayısı girilen bir diziyi sıralama verilebilir;

```
#include <iostream>
#include <algorithm> // std::sort
#include <vector>    // Modern dinamik dizi için
// Trailing return type kullanımı (Okunabilirlik için güzel bir tercih)
auto int_from_cin() -> int {
    int x;
    if (!(std::cin >> x)) return 0; // Hatalı girişe karşı basit bir koruma
    return x;
}
auto main() -> int {
    std::cout << "Girilen Tam Sayilari Siralama..." << std::endl;
    std::cout << "Kac adet sayi gireceksiniz: ";
    int const n = int_from_cin();
    // MODERN YAKLAŞIM: 'new int[n]' yerine 'std::vector' kullanımı.
    // Vector, kapsam dışına çıktığında belleği OTOMATİK temizler (delete[] gerektirmez).
    std::vector<int> a(n);
    for (int i = 0; i < n; ++i) {
        std::cout << i + 1 << ". elamani girin: ";
        a[i] = int_from_cin();
    }
    // std::sort artık vector'ün begin ve end iterator'ları ile çalışır
    std::sort(a.begin(), a.end());
    std::cout << "Siralanmis Dizi: ";
    for (int x : a) { // Modern 'range-based' for döngüsü
        std::cout << x << ' ';
    }
    std::cout << std::endl;
}
/*Program Çıktısı:
Girilen Tam Sayilari Siralama...
Kac adet sayi gireceksiniz: 5
1. elamani girin: 100
2. elamani girin: 10
3. elamani girin: 30
```

```
4. elamani girin: 20
5. elamani girin: 90
Siralanmis Dizi: 10 20 30 90 100
*/
```

## 12.7 explicit Sıfatı

C++ dilinde `explicit` sıfatı (explicit specifier), tiplerin üstü kapalı olarak dönüştürülmemesi için yapıcıları (constructor) nitelendirmek için kullanılır. Aşağıdaki kodu göz önüne alalım;

```
1  #include <iostream>
2  class Karmasik {
3  private:
4      double gercek;
5      double sanal;
6  public:
7      // Default parametreli yapıcı (Aynı zamanda Converting Constructor)
8      Karmasik(double r = 0.0, double i = 0.0) : gercek(r), sanal(i) {}
9      // İYİLEŞTİRME 1: Parametreyi 'const reference' (&) olarak almak kopyalamayı önler.
10     // İYİLEŞTİRME 2: Fonksiyonun sonuna 'const' eklemek, bu fonksiyonun nesneyi değiştirmediğini
       ↪ garanti eder.
11     bool operator==(const Karmasik& c) const {
12         return (gercek == c.gercek && sanal == c.sanal);
13     }
14 };
15 int main() {
16     Karmasik k(3.0, 0.0);
17     // std::cout ve std::endl kullanımı (namespace belirtilmediği için)
18     if (k == 3.0)
19         std::cout << "Ayni" << std::endl; // Çıktı bu olur.
20     else
21         std::cout << "Farkli" << std::endl;
22 }
```

Yukarıdaki örneği 12. satırındaki kontrol ifadesindeki `&&` işlecinin solundaki ifade doğru ise sağdakine bakılmaz. Bu nedenle program çıktısı `Aynı` olacaktır. Bunu önlemek için `explicit` anahtar kelimesi **yapıcı önünde kullanılır**.

```
    explicit Karmasik(double r = 0.0, double i = 0.0) : gercek(r), sanal(i) { }
```

Bu durumda program derlenince `no match for 'operator==' (operand types are 'Karmasik' and 'double')` derleme hatası alınır. Hala `double` değerlerini `Karmasik` tipe dönüştürebiliriz, ancak bu **durumda açıkça tip dönüşümü yapmamız gerekir**;

```
    if (k == (Karmasik) 3.0)
        std::cout << "Aynı";
    else
        std::cout << "Farklı";
```

## 12.8 Takma İsimler

C Dilinde halihazırda mevcut veri tiplerinin adını yeniden tanımlamak ya da **takma isim** (typedef) verilebilir;

```
typedef mevcutisim takmaisim;
```

Aşağıda takma isim verilmiş yeni veri tipleriyle yazılmış bir kod örneği verilmiştir;

```
#include <iostream>
typedef char karakter;
typedef int tamsayi;
typedef long long buyukolceklitamsayi;
typedef unsigned int pozitiftamsayi;
int main() {
    karakter k='A';
    tamsayi t=12;
    buyukolceklitamsayi bot=0x0FFFFFFF;
    pozitiftamsayi pt=1;
```

```
    std::cout << k << "," << t << "," << bot << "," << pt << std::endl;
}
```

Bunun dışında fonksiyonlara da takma isim verilebilir;

```
typedef int Fonk(int); /* Fonk bir fonksiyona verilen bir takma isimdir; Fonk, int tipinde
↪ parametre alan ve int geri döndüren her fonksiyon olabilir */
int faktoriyel(int n) { //faktoriyel fonksiyonu da Fonk tipindedir.
    return (n==1)? 1: n*faktoriyel(n-1);
}
int onKat(int i) {
    return i*10;
}
int main() {
    Fonk *fn; // fn bir Fonk tipinde olan fonksiyonlara olan göstericidir.
    fn = &faktoriyel; // faktoriyel fonksiyonunu göster
    fn(3); // 3! Hesaplanır
    fn = &onKat; // onKat fonksiyonunu göster
    fn(4); // 4*10 hesaplanır
}
```

Modern C++ standartlarında(C++11/14/17), `typedef` **anahtar kelimesi kullanılmaz**. Bunun yerine ileride göreceğimiz `using` anahtar kelimesi kullanılır.

## 12.9 using Anahtar Kelimesinin Takma İsim Olarak Kullanılması

`using` Anahtar kelimesi, 10.10.4 ve 11.12 başlıklarındaki kullanımı yanında, **takma ad** (type aliasing) olarak çok uzun veri tiplerini kısaca kullanımını sağlar;

```
#include <iostream>
#include <vector> //std::vector için
namespace alanadi{
    namespace icalanadi{
        class Sinif {
            private:
            int durum;
            public:
            Sinif(int pDurum):durum(pDurum) {}
            int getDurum() {return this->durum; }
        };
    }
}
// typedef yerine 'using' kullanımı (Modern C++ Yaklaşımı)
using karakter = char;
using tamsayi = int;
using buyukolceklitamsayi = long long;
using pozitiftamsayi = unsigned int;
using KisacaSinif = alanadi::icalanadi::Sinif;
int main() {
    KisacaSinif nesne(100);
    std::cout  << nesne.getDurum();
    karakter k = 'A';
    tamsayi t = 12;
    buyukolceklitamsayi bot = 0x0FFFFFFF;
    pozitiftamsayi pt = 1;
    std::cout << "k: " << static_cast<int>(k) << " veya " << k << std::endl;
    std::cout << "t: " << t << std::endl;
    // Onaltılık (hex) gösterimi istersek std::hex kullanabiliriz
    std::cout << "bot (desimal): " << bot << std::endl;
    std::cout << "bot (hex): 0x" << std::hex << std::uppercase << bot << std::dec << std::endl;
    std::cout << "pt: " << pt << std::endl;
    using StringList = std::vector<std::string>;
    StringList isimler; // std::vector<std::string> ile StringList aynıdır.
```

```
}
```

## 12.10 Düşün ve Çöz

- **Çöz**: `static_cast`, `dynamic_cast`, `const_cast` ve `reinterpret_cast` işleçlerini karşılaştırın. Her birinin güvenli ve uygun kullanım senaryolarını örneğiyle belirtin.
- **Düşün**: Daraltıcı (narrowing) dönüşüm neden tehlikeli olabilir? `int` veri tipinden `char` veri tipine dönüşümde veri kaybının yaşanacağı bir örnek gösterin ve modern C++ bunu nasıl önlemeye çalışır?
- **Düşün**: Lamda ifadelerinin söz dizimini açıklayın. Yakalama listesi `[=]`, `[&]` ve `[x]` arasındaki farkı bir örnekle gösterin. Kapatma (closure) kavramı nedir?
- **Düşün**: `auto` anahtar kelimesinin tip çıkarımını (type inference) nasıl gerçekleştirdiğini açıklayın. `auto` kullanımının kodun okunabilirliğini artırdığı ve azalttığı durumları örnekleyin.
- **Düşün**: `explicit` sıfatı olmayan ve olan yapıcı fonksiyonlar arasındaki farkı gösterin. Örtülü tip dönüşümü ne zaman istenmeyen sonuçlara yol açar?